

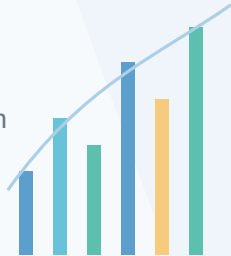


BIG DATA ANALYTICS FOR FINANCE

Lecture 1: Machine Learning and Python Review

Foundations, core models, evaluation, and hands-on practice

Dr. Nguyen Tuan Long National Economics University | 2026



Learning Objectives

Conceptual understanding

By the end of this lecture, you should be able to:

- ▶ Distinguish supervised, unsupervised, and reinforcement learning.
- ▶ Explain regression, classification, and clustering tasks.
- ▶ Recognize overfitting, underfitting, and data leakage.

Practical skills

You should also be able to:

- ▶ Train core models with Scikit-learn.
- ▶ Select appropriate evaluation metrics.
- ▶ Build a repeatable train–evaluate workflow.
- ▶ Interpret results in a finance-oriented context.

Understand

What the model learns

Implement

Fit, predict, evaluate

Interpret

Connect metrics to decisions

Lecture Roadmap

1. Foundations What ML is, learning types, tasks, and workflow



2. Supervised Models Linear/Logistic Regression, KNN, Trees, Forests



3. Evaluation Train-test split, errors, confusion matrix, ROC-AUC



4. Unsupervised Learning K-Means and customer segmentation



5. Practice Housing, tumor diagnosis, and retail RFM labs

PART 1

Machine Learning Foundations

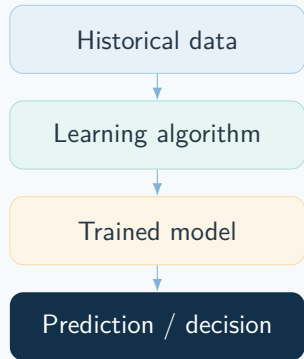
What Is Machine Learning?

Learn patterns from data instead of explicitly coding every rule.

- ▶ A model receives examples and adjusts its internal parameters.
- ▶ The learned pattern is used to make predictions on new data.
- ▶ Performance should improve with relevant data and appropriate feedback.

Traditional programming vs. ML

Rules + Data $\rightarrow$ Answers **vs.** Data +
Answers $\rightarrow$ Learned Rules



Three Main Learning Paradigms

Supervised	Unsupervised	Reinforcement
Data: features + labels ▶ House-price prediction ▶ Credit-risk classification ▶ Fraud detection Learn a mapping	Data: features only ▶ Customer segments ▶ Anomaly discovery ▶ Dimensionality reduction Discover structure	Data: actions + rewards ▶ Trading strategies ▶ Dynamic pricing ▶ Sequential decisions Learn by interaction

Focus of Lecture 1

Supervised learning receives most attention, followed by an introduction to K-Means clustering.

Regression, Classification, and Clustering

Regression

Predict a continuous value

salary, return, volatility, price

Classification

Predict a discrete class

fraud/not fraud, default/no default

Clustering

Discover similar groups

customer segments, behavior profiles

First question: What type of output does the business decision require?

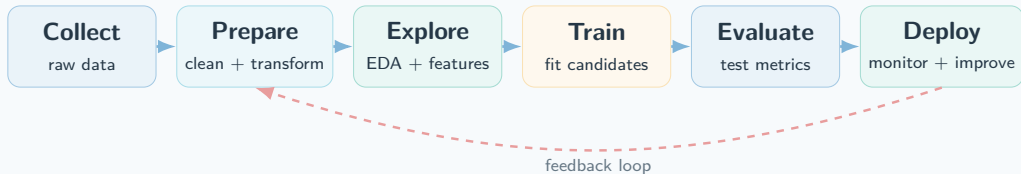
Core Models in This Lecture

Model	Task	Main idea
Linear Regression	Regression	Fit a line or hyperplane that minimizes prediction error.
Logistic Regression	Classification	Convert a linear score into a class probability.
K-Nearest Neighbors	Both	Predict from the labels or values of nearby observations.
Decision Tree	Both	Learn a sequence of interpretable if-then split rules.
Random Forest	Both	Aggregate many randomized trees for stability and accuracy.
K-Means	Clustering	Assign points to the nearest centroid and update centroids iteratively.

No universally best model

Model choice depends on the target, sample size, feature structure, interpretability needs, and cost of errors.

Standard Machine Learning Workflow



Data work dominates

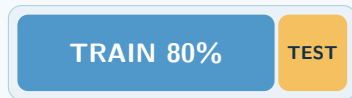
Cleaning, validation, feature design, and leakage prevention often matter more than changing algorithms.

The workflow is iterative

Evaluation frequently sends us back to the data, features, assumptions, or business objective.

Train–Test Split: A Fair Final Exam

- ▶ **Training set (70–80%):** used to learn model parameters.
- ▶ **Test set (20–30%):** held back for final evaluation.
- ▶ The test set represents observations the model has never seen.



Split before scaling or feature learning
to prevent information leakage

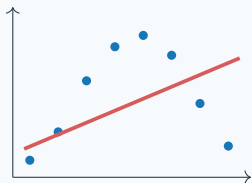
Generalization

The goal is not to memorize historical observations; it is to perform well on new observations from the same problem.

Reproducibility tip

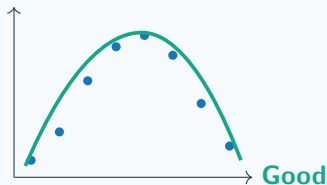
Use a fixed `random_state`; use `stratify=y` for classification when class proportions matter.

Underfitting vs. Overfitting



Underfitting

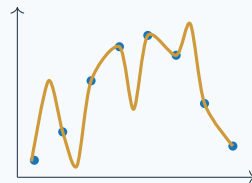
Too simple; poor train and test performance



fit

Good

Captures signal and generalizes



Overfitting

Memorizes noise; weak test performance

Control complexity with more data, regularization, depth limits, feature selection, and cross-validation.

PART 2

Supervised Learning: Regression

Regression Evaluation Metrics

Mean Squared Error

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Squares errors and strongly penalizes large misses.

Root Mean Squared Error

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Returns the error to the same unit as the target.

Mean Absolute Error

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Easier to interpret and less sensitive to outliers.

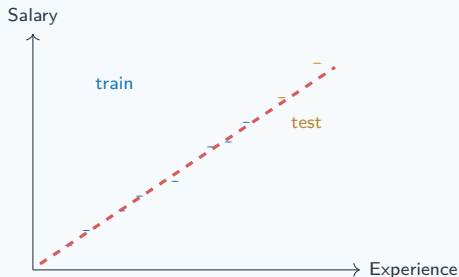
R^2 score

Compares the model with predicting the target mean. Closer to 1 is usually better; negative values are possible.

Running Example: Salary Prediction

Experience (years)	Salary (million)	Experience (years)	Salary (million)
1.0	15	5.0	35
1.5	18	5.5	36
2.5	22	6.0	40
3.0	25	7.0	45
4.0	28	8.0	52

- First 8 observations: training set.
- Final 2 observations: test set.



Manual Calculation: OLS Coefficients

Step 1: Calculate the sample means

$$\bar{x} = \frac{1 + 1.5 + \dots + 6}{8} = 3.5625$$
$$\bar{y} = \frac{15 + 18 + \dots + 40}{8} = 27.375$$

Step 2: Calculate the two sums

$$S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = 116.8125$$
$$S_{xx} = \sum (x_i - \bar{x})^2 = 24.21875$$

Step 3: Estimate slope and intercept

$$\beta_1 = \frac{S_{xy}}{S_{xx}} = \frac{116.8125}{24.21875} \approx 4.8232$$

$$\begin{aligned}\beta_0 &= \bar{y} - \beta_1 \bar{x} \\ &\approx 27.375 - (4.8232)(3.5625) \\ &\approx 10.1923\end{aligned}$$

Fitted equation

$$\widehat{Salary} = 10.1923 + 4.8232 \text{ Experience}$$

Manual Calculation: OLS Coefficients

Step 1: Calculate the sample means

$$\bar{x} = \frac{1 + 1.5 + \dots + 6}{8} = 3.5625$$
$$\bar{y} = \frac{15 + 18 + \dots + 40}{8} = 27.375$$

Step 2: Calculate the two sums

$$S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = 116.8125$$
$$S_{xx} = \sum (x_i - \bar{x})^2 = 24.21875$$

Step 3: Estimate slope and intercept

$$\beta_1 = \frac{S_{xy}}{S_{xx}} = \frac{116.8125}{24.21875} \approx 4.8232$$

$$\begin{aligned}\beta_0 &= \bar{y} - \beta_1 \bar{x} \\ &\approx 27.375 - (4.8232)(3.5625) \\ &\approx 10.1923\end{aligned}$$

Fitted equation

$$\widehat{Salary} = 10.1923 + 4.8232 \text{ Experience}$$

Linear Regression: Learn the Best-Fitting Line

Model

$$\hat{y} = \beta_0 + \beta_1 x$$

- ▶ β_0 : intercept or baseline prediction.
- ▶ β_1 : expected change in y for one unit of x .

The fitted salary example gives:

$$\widehat{Salary} = 10.19 + 4.82 \times Experience$$

Test predictions

$$x = 7 : \hat{y} \approx 43.95 \quad (y = 45)$$

$$x = 8 : \hat{y} \approx 48.78 \quad (y = 52)$$

$$RMSE \approx 2.395$$

Interpretation

In this small dataset, one extra year of experience is associated with about 4.82 million in salary.

Manual Calculation: Linear Regression Test Error

Step 1: Predict the two test observations

$$\begin{aligned}\hat{y}_7 &= 10.1923 + 4.8232(7) = 43.9548, \\ \hat{y}_8 &= 10.1923 + 4.8232(8) = 48.7781.\end{aligned}$$

Step 2: Calculate residuals

$$\begin{aligned}e_7 &= 45 - 43.9548 = 1.0452, \\ e_8 &= 52 - 48.7781 = 3.2219.\end{aligned}$$

Step 3: Square and average

$$\begin{aligned}MSE &= \frac{1.0452^2 + 3.2219^2}{2} \\ &= \frac{1.0924 + 10.3809}{2} = 5.7366\end{aligned}$$

Step 4: Return to salary units

$$RMSE = \sqrt{5.7366} = 2.3951$$

The typical test error is about 2.40 million.

The Scikit-learn Pattern

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np

# 1. Initialize
model = LinearRegression()

# 2. Learn from training data
model.fit(X_train, y_train)

# 3. Predict unseen observations
y_pred = model.predict(X_test)

# 4. Evaluate
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

Initialize

Choose model + settings

Fit

Learn from training data

Predict

Apply to new features

Evaluate

Measure decision quality

K-Nearest Neighbors Regression

1. Store the entire training set.
2. Measure distance from a new observation to training observations.
3. Select the K nearest neighbors.
4. Average their target values.

With $K = 3$, both test observations use salaries 40, 36, 35:

$$\hat{y} = \frac{40 + 36 + 35}{3} = 37$$

Result: $RMSE \approx 12.02$.

Strengths

Simple, flexible, and able to model nonlinear local patterns.

Limitations

Sensitive to feature scale, irrelevant features, K , and large datasets.

Key requirement

Scale features before using distance-based models.

Manual Calculation: KNN Regression ($K = 3$)

Test point: 7 years

Train x_i	Distance $ 7 - x_i $	Salary
6.0	1.0	40
5.5	1.5	36
5.0	2.0	35

$$\hat{y}_7 = (40 + 36 + 35)/3 = 37.$$

Test point: 8 years

The same three observations are nearest, so $\hat{y}_8 = 37$.

Calculate RMSE

$$e_7 = 45 - 37 = 8, \quad e_8 = 52 - 37 = 15$$

$$MSE = \frac{8^2 + 15^2}{2} = \frac{289}{2} = 144.5$$

$$RMSE = \sqrt{144.5} \approx 12.0208$$

Interpretation

KNN cannot extrapolate beyond the target values of nearby training observations.

Decision Tree Regression

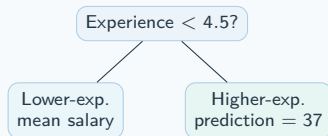
A tree repeatedly asks questions such as:

Experience < 4.5 years?

It selects the split that minimizes weighted error:

$$MSE_{split} = \frac{n_L}{n} MSE_L + \frac{n_R}{n} MSE_R$$

In the notebook example, the first split reduces MSE from 71.23 to 15.40.



Complexity control

Limit `max_depth`,
`min_samples_split`, or prune the
tree.

Manual Calculation: A Regression Tree Split

Candidate split: *Experience* < 4.5

Left salaries: 15, 18, 22, 25, 28

$$\bar{y}_L = 21.6$$

$$\begin{aligned} MSE_L &= \frac{43.56 + 12.96 + 0.16}{5} \\ &+ \frac{11.56 + 40.96}{5} \\ &= 21.84 \end{aligned}$$

Right branch

Right salaries: 35, 36, 40

$$\bar{y}_R = 37$$

$$\begin{aligned} MSE_R &= \frac{(35 - 37)^2 + (36 - 37)^2}{3} \\ &+ \frac{(40 - 37)^2}{3} \\ &= \frac{14}{3} = 4.67 \end{aligned}$$

Weighted error after the split

$$MSE_{split} = \frac{5}{8}(21.84) + \frac{3}{8}(4.67) = 15.40$$

The smallest weighted error among all candidate thresholds becomes the split.

Finding the Optimal Root Split

The model tests every midpoint between two consecutive experience values and selects the threshold with the lowest weighted error:

$$MSE_{\text{after split}} = \frac{n_{\text{left}}}{n_{\text{total}}} MSE_{\text{left}} + \frac{n_{\text{right}}}{n_{\text{total}}} MSE_{\text{right}}.$$

Split threshold (<)	MSE (Left)	MSE (Right)	MSE after split
1.25	0.00	55.27	48.36
2.00	2.25	33.56	25.73
2.75	8.22	26.24	19.48
3.50	14.50	18.69	16.60
4.50	21.84	4.67	15.40
5.25	41.81	4.00	32.35
5.75	47.63	0.00	41.68

Selected root rule

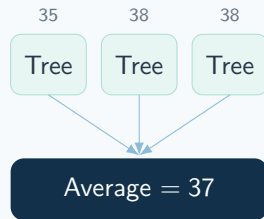
The model selects $Experience < 4.5$ because it reduces the root MSE from 71.23 to the minimum weighted MSE of 15.40. For the branch $Experience \geq 4.5$, the leaf prediction is

$$\hat{y}_{\text{right}} = \frac{35 + 36 + 40}{3} = 37.$$

Random Forest: Collective Intelligence

1. Draw multiple bootstrap samples from training data.
2. Train a tree on each sample using random feature subsets.
3. Average predictions for regression.
4. Use majority vote for classification.

Randomization reduces correlation among trees, while aggregation reduces variance.



Trade-off

Usually stronger and more stable than one tree, but less transparent and more computationally expensive.

Manual Calculation: Random Forest Aggregation

Suppose three bootstrapped trees predict the salary of a test observation as:

$$\hat{y}^{(1)} = 35, \quad \hat{y}^{(2)} = 38, \quad \hat{y}^{(3)} = 38$$

Random Forest regression averages the tree predictions:

$$\hat{y}_{RF} = \frac{35 + 38 + 38}{3} = 37$$

For the two test targets

$$y = 45 : (45 - 37)^2 = 64,$$

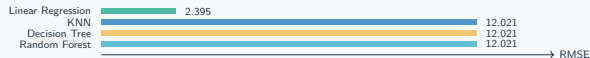
$$y = 52 : (52 - 37)^2 = 225.$$

$$RMSE = \sqrt{\frac{64 + 225}{2}} = 12.0208$$

Classification version

Replace the numerical average with a majority vote over class labels.

Regression Model Comparison



Result on this toy dataset

Linear Regression has the lowest test RMSE and therefore performs best.

Do not overgeneralize

The test set contains only two observations. The comparison illustrates mechanics, not universal model rankings.

PART 3

Supervised Learning: Classification

Classification Example: Pass or Fail

Study hours	Result	Study hours	Result
1.0	Fail	5.5	Pass
2.0	Fail	6.0	Pass
3.0	Fail	7.0	Pass
4.5	Fail	8.0	Pass
4.0	Fail	9.0	Pass

First 8 rows are training data; highlighted rows are test data.

Target

- ▶ Fail = 0
- ▶ Pass = 1

Basic metric

$$Accuracy = \frac{\text{correct predictions}}{\text{total predictions}}$$

Logistic Regression: From Score to Probability

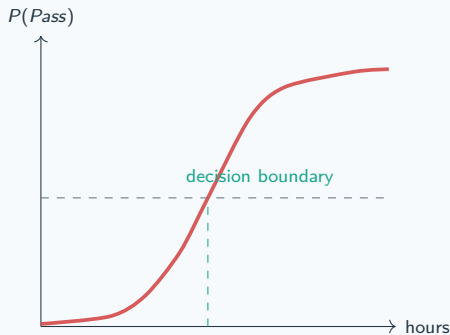
Sigmoid function

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

The fitted notebook example gives:

$$\beta_0 \approx -5.5288, \quad \beta_1 \approx 1.1590$$

A common rule is: predict Pass if probability > 0.5 .



$$x = 4 : P(\text{Pass}) \approx 0.29 \Rightarrow \text{Fail}$$

$$x = 9 : P(\text{Pass}) \approx 0.99 \Rightarrow \text{Pass}$$

Manual Calculation: Logistic Probabilities

The fitted score is $z = -5.5288 + 1.1590x$, and $P(\text{Pass}) = 1/(1 + e^{-z})$.

Student studying 4 hours

$$z_4 = -5.5288 + 1.1590(4) = -0.8928,$$

$$P(\text{Pass}) = \frac{1}{1 + e^{0.8928}} \\ \approx 0.2905.$$

Since $0.2905 < 0.5$, predict **Fail**.

Student studying 9 hours

$$z_9 = -5.5288 + 1.1590(9) = 4.9022,$$

$$P(\text{Pass}) = \frac{1}{1 + e^{-4.9022}} \\ \approx 0.9926.$$

Since $0.9926 > 0.5$, predict **Pass**.

Accuracy on the two test observations

Both predictions are correct: $\text{Accuracy} = 2/2 = 100\%$.

KNN, Decision Tree, and Random Forest Classifiers

KNN	Decision Tree	Random Forest
Neighbor voting with $K = 3$: ▶ 4 hours → Fail ▶ 9 hours → Pass Local similarity	Best first split: $hours < 5.0?$ This creates two pure groups in the toy dataset. If-then rules	Multiple trees vote for the final class, improving stability over a single tree. Ensemble vote

Toy result

All four classifiers achieve 100% accuracy on two clearly separable test observations.

Manual Calculation: KNN Classification Vote

Query: 4 study hours

Neighbor	Distance	Label
4.5	0.5	Fail
3.0	1.0	Fail
5.5	1.5	Pass

Vote : 2 Fail vs. 1 Pass $\Rightarrow$ Fail

Query: 9 study hours

Neighbor	Distance	Label
8.0	1.0	Pass
7.0	2.0	Pass
6.0	3.0	Pass

Vote : 3 Pass vs. 0 Fail $\Rightarrow$ Pass

Both test labels are predicted correctly.

How a Classification Tree Chooses a Split

Gini impurity

$$Gini = 1 - \sum_{i=1}^C p_i^2$$

$Gini = 0$ means a completely pure node.

At the root: 4 Fail and 4 Pass observations.

$$Gini = 1 - (0.5^2 + 0.5^2) = 0.5$$

Weighted split impurity

$$Gini_{split} = \frac{n_L}{n} Gini_L + \frac{n_R}{n} Gini_R$$

The threshold $hours < 5.0$ produces:

- ▶ Left node: all Fail
- ▶ Right node: all Pass
- ▶ $Gini_{split} = 0$

Manual Calculation: Gini at $hours < 5.0$

Left child: 4 Fail, 0 Pass

$$p_{Fail} = 4/4 = 1, \quad p_{Pass} = 0/4 = 0$$
$$Gini_L = 1 - (1^2 + 0^2) = 0$$

Right child: 0 Fail, 4 Pass

$$p_{Fail} = 0/4 = 0, \quad p_{Pass} = 4/4 = 1$$
$$Gini_R = 1 - (0^2 + 1^2) = 0$$

A Gini value of zero means the node is pure.

This node is also completely pure.

Weighted Gini after the split

$$Gini_{split} = \frac{4}{8}(0) + \frac{4}{8}(0) = 0$$

Impurity falls from 0.5 at the root to 0, so this is the best possible split.

When Should Features Be Scaled?

Model	Scale?	Why?
KNN	Yes	Distance is dominated by features with large numerical ranges.
Linear Regression	Usually	Helps optimization and coefficient comparability, especially with regularization.
Logistic Regression	Usually	Improves numerical optimization and regularized models.
Decision Tree	No	Splits compare one feature with a threshold at a time.
Random Forest	No	Tree-based rules are insensitive to monotonic rescaling.
K-Means	Yes	Cluster assignment is based on distance to centroids.

Leakage warning

Fit the scaler on **training data only**; use its learned parameters to transform the test set.

PART 4

Evaluation Beyond Accuracy

Why Accuracy Can Be Misleading

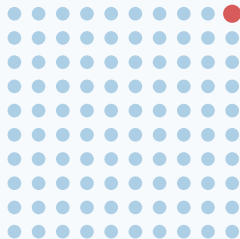
Imagine a rare disease dataset:

- ▶ 99% normal cases
- ▶ 1% sick cases

A model that always predicts “normal” achieves:

$$\text{Accuracy} = 99\%$$

Yet it detects **zero** sick patients.



One positive case can be hidden inside an apparently excellent score.

Decision principle

Choose metrics based on the business cost of false alarms and missed cases, not convenience.

Confusion Matrix

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

- ▶ **TP:** correctly detected positive.
- ▶ **TN:** correctly detected negative.
- ▶ **FP:** false alarm.
- ▶ **FN:** missed positive case.

Finance example

In fraud detection, an FN is a fraudulent transaction incorrectly approved as normal.

Precision, Recall, and F1-Score

Precision	Recall	F1-Score
$\frac{TP}{TP + FP}$ Of all positive predictions, how many were correct? Reduce false alarms	$\frac{TP}{TP + FN}$ Of all actual positives, how many were detected? Reduce missed cases	$2 \frac{Precision \cdot Recall}{Precision + Recall}$ Harmonic mean when both objectives matter. Balance both

ROC Curve and AUC



- ▶ ROC evaluates performance across probability thresholds.
- ▶ It plots True Positive Rate against False Positive Rate.
- ▶ AUC summarizes ranking quality across the entire curve.

Interpretation

AUC = 0.5: random ranking.
AUC closer to 1: stronger separation.

PART 5

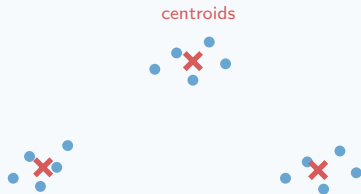
Unsupervised Learning and K-Means

K-Means Clustering

1. Choose K initial centroids.
2. Assign each point to its nearest centroid.
3. Recalculate each centroid as the cluster mean.
4. Repeat until assignments stabilize.

Finance applications

Customer segmentation, anomaly exploration, behavioral profiles, and portfolio grouping.



Important

K must be selected in advance and results depend on scaling and initialization.

K-Means with Scikit-learn

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X, _ = make_blobs(
    n_samples=300,
    centers=3,
    cluster_std=0.60,
    random_state=42
)

kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init="auto"
)

labels = kmeans.fit_predict(X)
centroids = kmeans.cluster_centers_
```

Model outputs

- ▶ Cluster label for each row
- ▶ Coordinates of centroids
- ▶ Inertia / within-cluster sum of squares

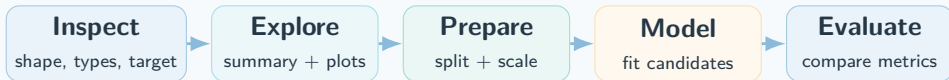
Elbow method

Plot inertia for several values of K and look for the point where improvement begins to flatten.

PART 6

Hands-on Practice

Lab Workflow



Notebook convention

Complete every YOUR CODE HERE cell and preserve reproducibility with `random_state=42`.

Expected evidence

Code, metrics, visualizations, and a short explanation connecting results to model behavior.

Lab 1: California Housing Regression

1. Load `fetch_california_housing`.
2. Build a `DataFrame` and inspect distributions.
3. Set features X and target `MedHouseVal`.
4. Split 80:20 and fit the scaler on training data only.
5. Compare Linear Regression, KNN, Decision Tree, and Random Forest using RMSE.

Questions

- ▶ Which model has the lowest RMSE?
- ▶ Why might a nonlinear model outperform a linear model?
- ▶ Does the best result justify extra complexity?

Visualization

Plot actual vs. predicted values and a horizontal comparison of RMSE scores.

Lab 2: Breast Cancer Classification

1. Load `load_breast_cancer`.
2. Inspect feature values and class balance.
3. Split with `stratify=y`; scale using training data.
4. Fit Logistic Regression, KNN, Decision Tree, and Random Forest.
5. Compare Accuracy, Precision, Recall, confusion matrices, and AUC.

Decision question

Which is more important for malignant tumors: Precision or Recall?

Recommended answer

Recall is critical because a false negative means a malignant case was incorrectly classified as benign.

Lab 3: Customer Segmentation with RFM

Create customer-level features

- ▶ **Recency:** days since last purchase.
- ▶ **Frequency:** number of unique invoices.
- ▶ **Monetary:** total customer spending.

Clean missing customer IDs, negative quantities, and invalid prices before aggregation.

Business interpretation

Translate clusters into strategies such as loyal customers, big spenders, new customers, and at-risk customers.

1. Standardize RFM features.
2. Use the elbow method for $K = 1, \dots, 10$.
3. Fit K-Means with the selected K .
4. Profile cluster averages.
5. Name actionable segments.

Advanced Exercises

Hyperparameter experiments

- ▶ Try KNN with $K = 3, 7, 9$.
- ▶ Increase tree depth and compare train vs. test error.
- ▶ Increase the number of trees in Random Forest.
- ▶ Explain whether performance changes are stable.

Evaluation extensions

- ▶ Plot ROC curves for all classifiers together.
- ▶ Compare AUC and recall, not only accuracy.
- ▶ Inspect feature importance from tree models.
- ▶ Identify possible leakage or unfair comparisons.

Explain every result; do not report metrics without interpretation.

Key Takeaways

- ▶ Start with the business target and task type.
- ▶ Separate training and test data before learning transformations.
- ▶ Match model behavior to data structure and constraints.
- ▶ Scaling is essential for distance-based models.
- ▶ Evaluate regression with error-based metrics.
- ▶ Evaluate classification using costs, not accuracy alone.
- ▶ Control model complexity to improve generalization.
- ▶ Interpret results before recommending action.

Data

Reliable and leakage-free

Model

Appropriate complexity

Metric

Aligned with decisions

References and Next Steps

Primary resources

- ▶ Scikit-learn Getting Started and User Guide.
- ▶ Course notebook: `L01_ML_Python_Review_EN.ipynb`.
- ▶ Python for Data Science learning materials from National Economics University.

Before the next session

Complete the three practice problems and retain all code, figures, and written interpretations.

Coming next

Data preparation: missing values, outliers, encoding, scaling, imbalanced data, and feature selection.

Questions?